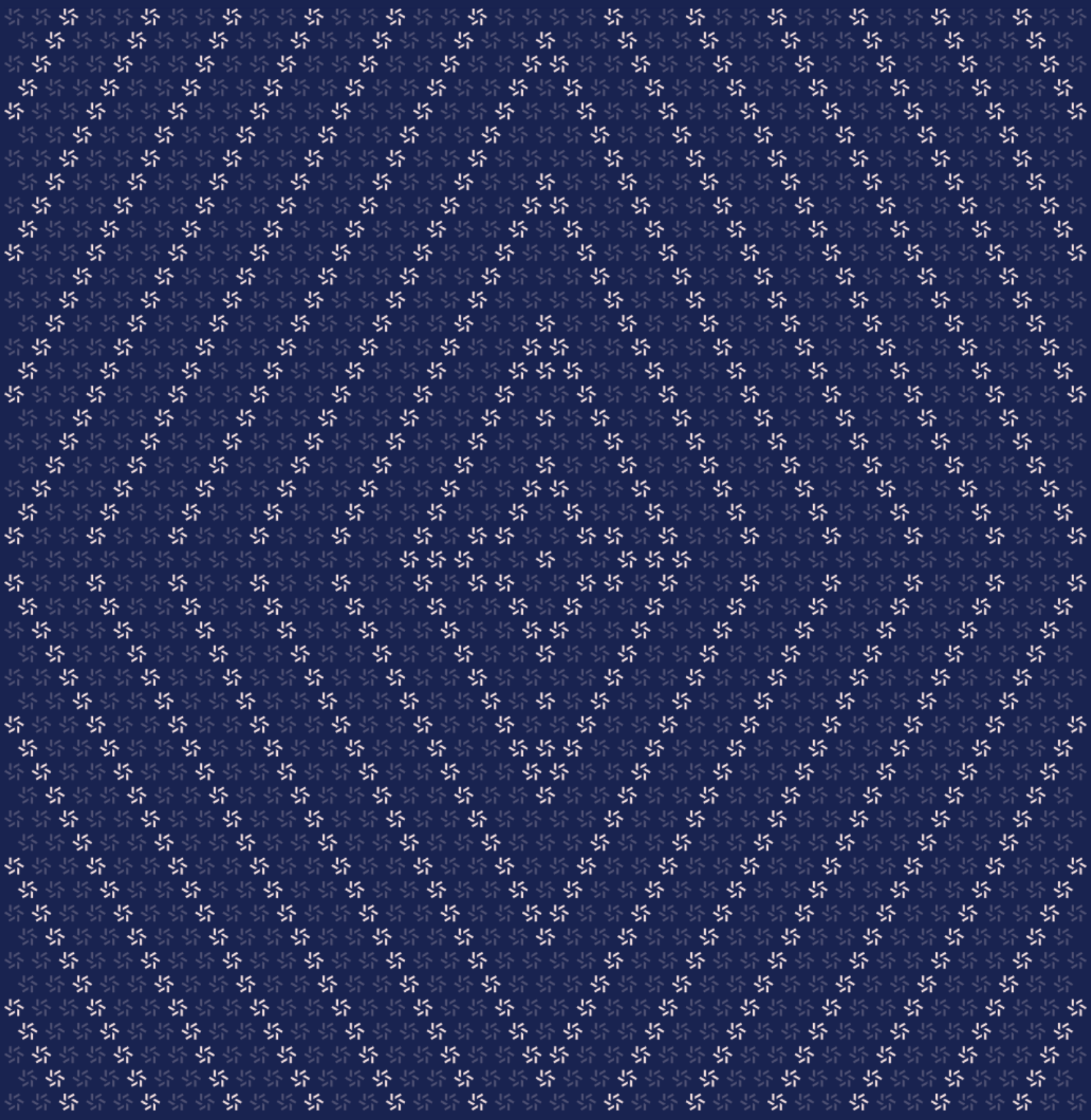
Zellic

Prepared for
Mark from Denmark
JPThor
THORChain

Prepared by
Seunghyeon Kim
Junyi Wang
Zellic

November 21, 2024

THORChain Bifrost
Program Security Assessment



Contents

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow @zellic_io on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for THORChain from October 17th to November 11th, 2024. During this engagement, Zellic reviewed THORChain Bifrost's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Does THORChain identify events from its own smart contracts properly?
- Is it possible for an attacker to synthetically cause an event to be detected?
- Is there any way to cause a transaction to be repeated?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- THORChain Cosmos-related components
- Configuration of the different server processes
- LevelDB internals
- Key custody

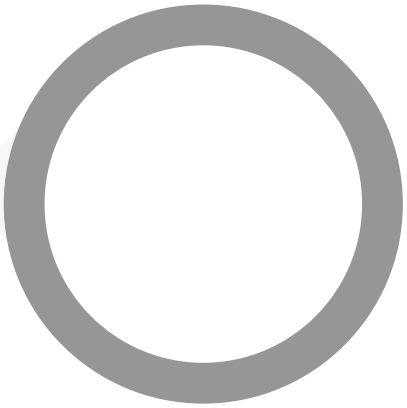
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped THORChain Bifrost contracts, we discovered two findings, both of which were informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	0
Informational	2



2. Introduction

2.1. About THORChain Bifrost

THORChain contributed the following description of THORChain Bifrost:

> THORChain is secured by its native token, RUNE, which deterministically accrues value as more assets are deposited into the network.
>
> Anyone can use THORChain to swap native assets between any supported chains or deposit their assets to earn yield from swaps.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case

basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

THORChain Bifrost Contracts

Type	Go
Platform	EVM-compatible

Target	thornode
Repository	https://gitlab.com/thorchain/thornode ↗
Version	27cc6e64fae3165280d9bdd82c98344208e768c2c
Programs	ethereum/ethereum.go ethereum/ethereum_block_scanner.go ethereum/fee_config.go ethereum/smart_contract.go ethereum/unstuck.go ethereum/whitelist_smartcontract.go ethereum/whitelist_smartcontract_aggregators.go evm/client.go evm/evm.go evm/evm_block_scanner.go evm/helpers.go evm/unstuck.go shared/evm/block_meta_accessor.go shared/evm/eth_rpc.go shared/evm/key_sign_wrapper.go shared/evm/leveldb_block_meta_accessor.go shared/evm/smart_contract.go shared/evm/smartcontract_log_parser.go shared/evm/token.go shared/evm/token_manager.go shared/evm/tokens_db.go shared/evm/utils.go shared/runners/solvency.go shared/signercache/signer_cache_mgr.go shared/signercache/signer_cache_store.go

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 5.4 person-weeks. The assessment was conducted by two consultants over the course of 5 calendar weeks.

Contact Information

The following project managers were associated with the engagement:

Jacob Goreski
Engagement Manager
jacob@zellic.io ↗

Chad McDonald
Engagement Manager
chad@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Seunghyeon Kim
Engineer
seunghyeon@zellic.io ↗

Junyi Wang
Engineer
junyi@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

October 17, 2024	Kick-off call
October 17, 2024	Start of primary review period
November 11, 2024	End of primary review period
TBD	Closing call

3. Detailed Findings

3.1. The function `isToValidContractAddress` can be optimized

Target	evm_block_scanner, ethereum_block_scanner		
Category	Optimization	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The `isToValidContractAddress` function can be optimized. In the below code, a contract address is checked against a whitelist.

```
// isToValidContractAddress this method make sure the transaction to address is
    to
// THORChain router or a whitelist address
func (e *EVMScanner) isToValidContractAddress(addr *ecommon.Address,
    includeWhiteList bool) bool {
    if addr == nil {
        return false
    }
    // get the smart contract used by thornode
    contractAddresses := e.pubkeyMgr.GetContracts(e.cfg.ChainID)
    if includeWhiteList {
        contractAddresses = append(contractAddresses, e.whitelistContracts...)
    }

    // combine the whitelist smart contract address
    for _, item := range contractAddresses {
        if strings.EqualFold(item.String(), addr.String()) {
            return true
        }
    }
    return false
}
```

The code unnecessarily appends the `whitelistContracts` to the created list.

Recommendations

Consider checking the whitelisted contracts in a separate loop.

Remediation

TBD

3.2. One LevelDB query can be omitted

Target	leveldb_block_meta_accessor		
Category	Optimization	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

A LevelDB query can be eliminated by handling the error from `Get`.

The below code queries LevelDB.

```
func (t *LevelDBBlockMetaAccessor) GetBlockMeta(height int64)
    (*evmtypes.BlockMeta, error) {
    key := t.getBlockMetaKey(height)
    exist, err := t.db.Has([]byte(key), nil)
    if err != nil {
        return nil, fmt.Errorf("fail to check whether block meta(%s) exist:
    %w", key, err)
    }
    if !exist {
        return nil, nil
    }
    v, err := t.db.Get([]byte(key), nil)
    if err != nil {
        return nil, fmt.Errorf("fail to get block meta(%s) from storage: %w",
    key, err)
    }
    var blockMeta evmtypes.BlockMeta
    if err = json.Unmarshal(v, &blockMeta); err != nil {
        return nil, fmt.Errorf("fail to unmarshal block meta from json: %w",
    err)
    }
    return &blockMeta, nil
}
```

One can check for an error from `Get` that indicates the key does not exist, rather than checking `Has` first.

Recommendations

Consider implementing the optimization.

Remediation

TBD

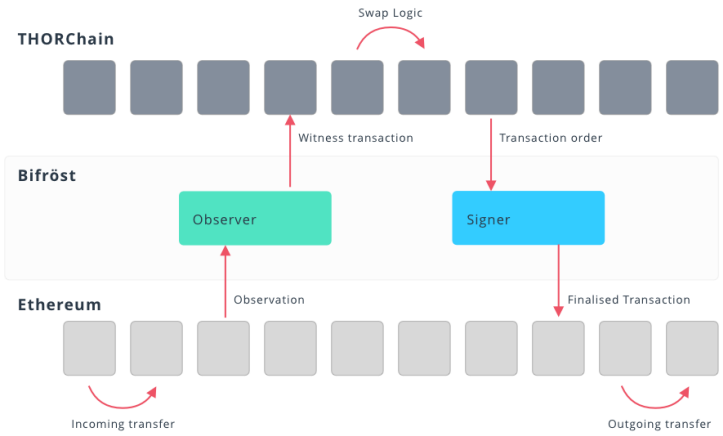
4. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

4.1. Bifrost

Bifrost of THORChain is consistent with Observer and Signer.



Observer is responsible for scanning blocks in the external chain, detecting new transactions, and forwarding them to THORChain.

Signer receives a remittance request from THORChain, signs the transaction, and sends it to an external chain. The Client structure performs this function, especially code related to `SignTx`.

4.2. Observer

This role is handled by code related to `ETHScanner`.

`ETHScanner`

The `ETHScanner` is a part of the Bifrost module. This interacts with the Ethereum chain and extracts the block and transactions, which is required for THORChain.

Here are the defined methods and what they do:

- `NewETHScanner` — This is the constructor of the `ETHScanner`.
- `GetGasPrice` — This returns the gas price. If the gas price is not set, it returns the cached gas price.
- `GetHeight` — This gets the block height from Ethereum.
- `FetchTxs` — This gets the transactions of the specific block height. This processes the block with the method `processBlock` and saves required metadata.
- `updateGasPrice` — This calculates the base and priority gas rates for the latest block and adds them to `gasCache`.
- `processBlock` — This extracts and processes transactions from a given block and returns them in `TxIn` format.
- `extractTxs` — This processes in-block transactions in parallel to form a `TxIn` object.
- `getTxInFromSmartContract` — This analyzes transactions made through smart contracts and converts them to `TxInItem`.
- `getTxInFromTransaction` — This processes general transactions and configures `TxInItem`.
- `getTxInFromFailedTransaction` — This is used to report failed transactions to THORNode due to lack of gas.

The `ETHScanner` supports seamless interaction between THORChain and the Ethereum network, efficiently collecting, analyzing, and processing transactions occurring on the Ethereum network.

4.3. Signer

The signing process begins in the `SignTx` function of the corresponding chain. Its interface is defined in the ChainClient interface as follows.

```
type ChainClient interface {
    // [...]
    SignTx(tx types.TxOutItem, height int64) ([]byte, []byte, *types.TxInItem,
    error)
```

The signing function is passed a transaction from THORChain and figures out the correct equivalent transaction on the corresponding chain before returning the signed transaction on success.

Specifically, it performs the following on the EVM chains (including Ethereum):

- Finds the address of the vault listed in the transaction
- Figures out the correct function/method to call on the vault smart contract based on the type of the transaction
- Finds the appropriate nonce to use and checks there are not too many transactions already pending
- Computes an appropriate gas base price and tip, as well as the max gas
- Creates and signs the transaction object, determining if a regular signature or TSS signature is required, depending on the vault being signed for

There are some slight differences between the Ethereum version and the EVM version, namely that the construction of the transaction, including the determination of the function/method and smart contract address, has been delegated to the `buildOutboundTx` method, which further delegates to the `getOutboundTxData` method.

5. Assessment Results

At the time of our assessment, the reviewed code was deployed.

During our assessment on the scoped THORChain Bifrost contracts, we discovered two findings, both of which were informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.